Branching Model:

```
main        → stable, finished code only
dev         → integration branch (where features merge)
feature/*   → individual work branches
```

Main:

- Used for demos, submissions, and release
- DON'T push to main directly

Dev

- Where we push our completed features
- It's fine to mess up here a little

Feature

- Each person's personal branches when they are adding things to our app
- One feature per branch. Ex: Nav Bar for our app gets one feature
- Ex: feature/loginPage

Naming Rules for Branches

Name your features like so:

```
feature/login
feature/api-endpoints
feature/ui-navbar
feature/data-pipeline
```

Git Workflow (I wrote bullet points generally explaining what the commands do)

1. Starting New Work

```
git checkout dev
git pull origin dev
git checkout -b feature/your-feature-name
```

- **Switch into the dev branch**
- **Update your local repo to match whatever is on the remote repo**
- **Create your new feature branch and switch into it**

2. Work and Commit

- **Add your changes to the staging area**
- **Commit your changes to your own feature branch. Please make sure you are not working directly in the dev branch or the main branch.**
- **Also, write clear, concise commit messages**

```bash
git add .
git commit -m "Add user authentication logic"
```

3. Push your Changes

```
git push origin feature/your-feature-name
```

Rules for Pull Requests

No direct merges.

When someone finishes a feature:

- **Go to the remote GitHub repo**
- **Open a Pull Request**
    - **Base: `dev`**
    - **Compare: `feature/your-feature-name`**
- **Have at least 1 teammate review before merging (We can use WhatsApp to coordinate this well!!)**
- **Resolve conflicts if needed**
- **Merge into `dev (NEVER MAIN!!)`**

`We should only push to main once we have reached big milestones`

`Example: home page completed and functional`

For `dev`:

- **Require a pull request before merging**
- **Require at least 1 approval**